

Università degli Studi di Milano Bicocca
Scuola di Scienze
Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di Laurea in Informatica

Appunti Advanced Machine Learning

Anno Accademico 2026-2027

Contents

1 Introduction to Deep Learning **2**
- 1.1 Representation learning . . . 2
- 1.2 Deep learning . . . 3

2 Linear Models **4**
- 2.1 Score Function of Linear Models . . . 4
- 2.2 Output And Loss Function of Linear Models . . . 5
  - 2.2.1 Regression . . . 5
  - 2.2.2 Binary Classification (Hinge) . . . 6
  - 2.2.3 Binary Classification (Logistic) . . . 7
  - 2.2.4 Multi-class Classification . . . 8
- 2.3 XOR Problem in Linear Model . . . 9

3 Deep Feedforward Networks **11**
- 3.1 Anatomy of a Unit and of a Layer . . . 11
- 3.2 Multilayer Network in Matrix Form . . . 12
- 3.3 Solving XOR with NN . . . 13
- 3.4 Hidden Unit as Feature Detectors . . . 14
- 3.5 Cost Functions and Output Units . . . 15
  - 3.5.1 Gradient-Based Learning . . . 15
- 3.6 Notation . . . 15

4 Exercises **17**

Chapter 1

Introduction to Deep Learning

Deep learning is a specialized subset of machine learning centered on multilayer neural networks, whose architectures are conceptually inspired by the structure of the human brain. Rather than relying on hand-crafted features, deep learning operates within the paradigm of *representation learning*, enabling models to automatically discover hierarchies of abstract features built from simpler ones. Today, deep learning underpins many of the most advanced artificial intelligence (AI) systems, driving state-of-the-art applications ranging from computer vision and generative AI to autonomous driving and robotics. As illustrated in Figure 1.1, deep learning represents a specific, highly expressive branch within the broader scope of artificial intelligence.

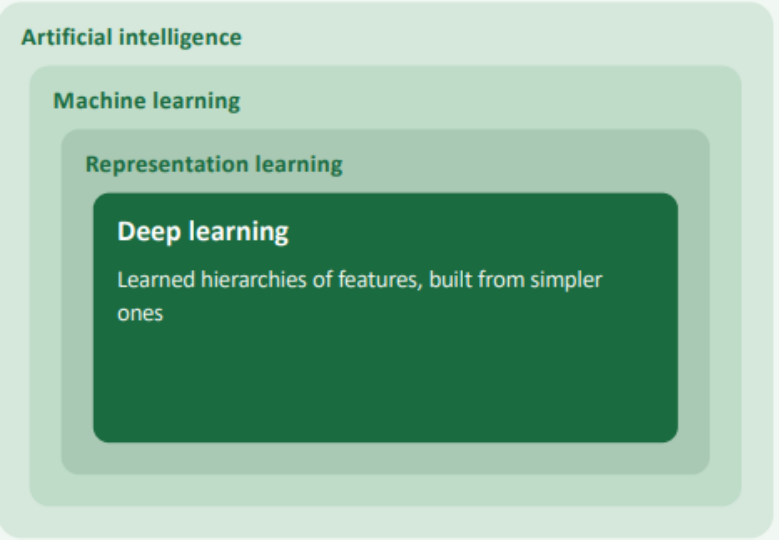


Figure 1.1: Hierarchical positioning of Deep Learning

1.1 Representation learning

What really distinguishes deep learning from traditional machine learning is its ability to perform *representation learning*. In traditional machine learning, the model is provided with a set of

features that are manually engineered by domain experts. In contrast, deep learning models automatically learn to extract relevant features from raw data with representation learning. In figure 1.2 we can see four different approaches to the task of representation. The main goal of

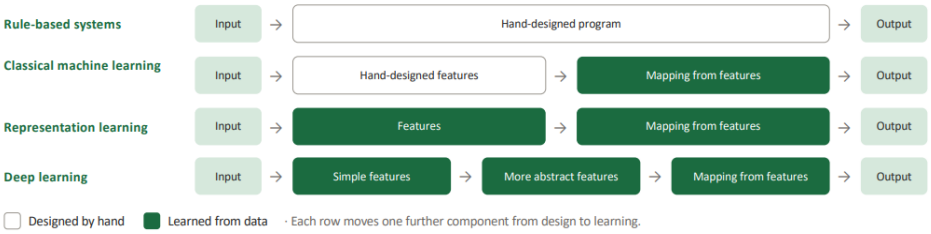


Figure 1.2: Four approaches to representation learning

representation learning is to identify the **factor of variations** that explain the observed data (that sometimes are not directly observable). They might be seen as concepts or abstraction that help us make sense of the rich variability in the data.

1.2 Deep learning

Deep learning uses a hierarchical approach to representation learning, allowing models to obtain features by introducing representations that are expressed in terms of other simpler representations. These architectures frequently leverage *distributed representations*, where concepts are encoded through the joint, many-to-many activity of multiple processing units rather than isolated nodes, significantly improving efficiency and generalization.

Chapter 2

Linear Models

In order to understand the power and the importance of Feedforward Networks, we first need to understand the limitations of linear models. Linear models are a class of models that make predictions based on a linear combination of the input features. They are simple and efficient, but they have limitations in capturing complex relationships in data. In Table 2.1, we summarize the main types of linear models, their output functions, output ranges, and loss functions.

Task	**Score**	**Output function**	**Output range**	**Loss**
Regression	$z = \mathbf{w}^\top \mathbf{x} + b$	identity: $\hat{y} = z$	$\mathbb{R}$	$(\hat{y} - y)^2$
Binary (hinge)	$z = \mathbf{w}^\top \mathbf{x} + b$	$\hat{y} = \text{sign}(z)$	$\{-1, +1\}$	$\max(0, 1 - yz)$
Binary (logistic)	$z = \mathbf{w}^\top \mathbf{x} + b$	$\hat{y} = \sigma(z)$	$(0, 1)$	$-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$
Multi-class	$z = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$	$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$	probability vectors	$-\log \hat{y}_y$

Table 2.1: Models with a Linear Score

2.1 Score Function of Linear Models

Every task solved with a linear model relies on a score function that computes a linear combination of the input features. The score (often denoted as z or referred to as *logits* in classification) represents the initial, raw evidence or confidence calculated by the model before any final mapping. For regression and binary classification tasks, given a single input vector $\mathbf{x} \in \mathbb{R}^d$, the score is a scalar computed as a weighted sum plus a bias:

$$z = \mathbf{w}^\top \mathbf{x} + b \tag{2.1}$$

Geometrically, this score defines a single hyperplane in the input space that serves as the decision boundary ($z = 0$) or predicts a continuous value (in the case of regression).
For multi-class tasks with k classes, the model computes a vector of scores rather than a single scalar. Using matrix notation for a weight matrix $\mathbf{W}$ and a bias vector $\mathbf{b}$:

$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} \tag{2.2}$$

In this multi-class setting, the model evaluates multiple scores simultaneously, where the boundaries between classes are formed where individual score components equate ($z_i = z_j$).

2.2 Output And Loss Function of Linear Models

The output and loss functions are different for each task, the first maps the raw score to the final prediction and the second measures the error between the prediction $\hat{y}$ and the true target y in order to be able to update the model parameters.

2.2.1 Regression

In regression, the output function is the identity function, meaning the raw score is used directly as the prediction:

$$\hat{y} = z \tag{2.3}$$

So the output range is the entire set of real numbers ($\mathbb{R}$). In Fig. 2.1 there is an example of a linear regression in 2d (one feature).

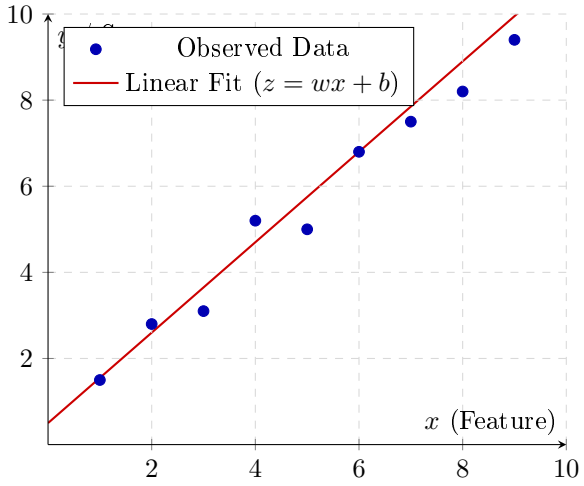


Figure 2.1: Geometric representation of 2D linear regression.

The loss function for regression is the **Squared Error**:

$$l(\hat{y}, y) = (\hat{y} - y)^2 \tag{2.4}$$

that means that the loss grows quadratically with the error, which can be sensitive to outliers. so it's often used the **Absolute Error**:

$$l(\hat{y}, y) = |\hat{y} - y| \tag{2.5}$$

that grows only linearly.
In figure 2.2 we can see a comparison between the two loss functions if we take tre example ($\hat{y}$,y): (3,5;3)(2;3)(5;3). As we can se the squared error is geometrically a parabola and the absolute error is a V shape.

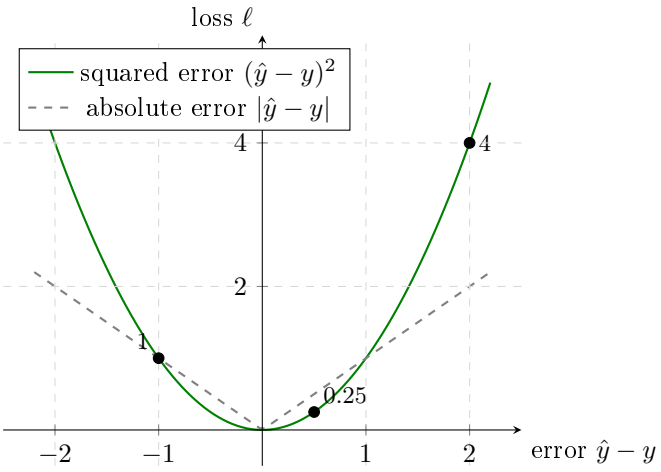


Figure 2.2: Comparison between squared error loss and absolute error loss for linear regression.

2.2.2 Binary Classification (Hinge)

In this type of classification task (e.g Support Vector Machines), the output function is the sign function, which maps the raw score to either -1 or +1:

$$\hat{y} = \text{sign}(z) \tag{2.6}$$

where

$$\text{sign}(z) = \begin{cases} +1 & \text{if } z > 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{if } z < 0 \end{cases}$$

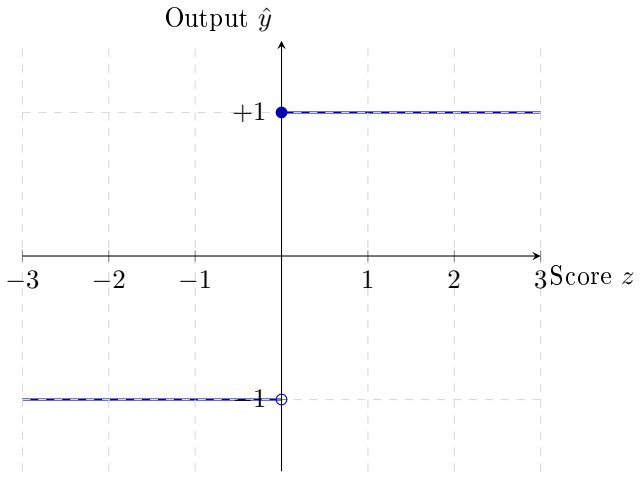


Figure 2.3: Output function for SVM / Hinge loss classification: $\hat{y} = \text{sign}(z)$.

The loss function for binary classification with hinge loss is defined as:

$$l(\hat{y}, y) = \max(0, 1 - yz) \tag{2.7}$$

This function identifies three cases:

1. $yz < 0$: the prediction is wrong and the loss grows linearly.
2. $0 \leq yz < 1$: the prediction is correct, but it is too close to the decision boundary.
3. $yz \geq 1$: the prediction is correct and sufficiently far from the boundary, so the loss is zero.

In the $2^n d$ case we have a value of z equal to 0 or close to it, so a very small variation (i.e. a noise) could lead to the opposite classification, so the loss isn't 0 even if the prediction is correct. This is the geometric intuition behind margin maximization: the model is penalized only when the prediction is either wrong or not confident enough. In Fig 2.4 there is a raffiguration of what written above if we consider three cases of (y;z): (+1;3)(+1;0,4)(-1;2).

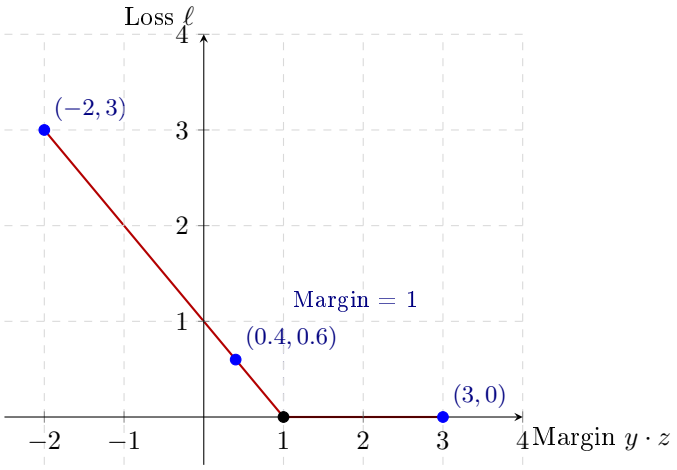


Figure 2.4: Geometric representation of the Hinge Loss function with specific examples.

2.2.3 Binary Classification (Logistic)

In this task (e.g. Logistic Regression) the goal is to predict a discrete or categorical outcome. The output function is the **sigmoid**:

$$\hat{y} = \sigma(z) \tag{2.8}$$

where σ is:

$$\sigma = \frac{1}{1 + e^{-z}}$$

which maps the raw score in the range of [0,1], as shown in Fig 2.5

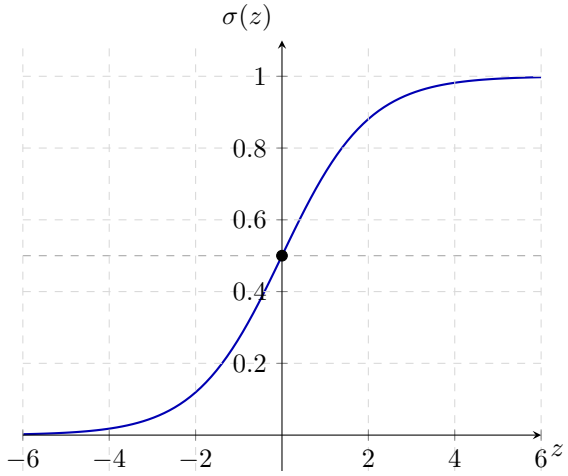


Figure 2.5: Geometric representation of the sigmoid function.

The loss fuction is the **Binary Cross-Entropy Loss**:

$$\mathcal{L}(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}) \tag{2.9}$$

The model outputs a probability that the class is 1, so when z is large the loss is small; The smaller zeta becomes, the greater the loss becomes. For the class $y = 0$ is mirrored. If we look at the formula 2.2.3, the loss can't be really 0.

2.2.4 Multi-class Classification

In multi-class classification tasks (e.g., Softmax Regression / Multinomial Logistic Regression), the model predicts a probability distribution across C discrete classes. The output function is the **softmax** function, which maps a vector of raw scores $\mathbf{z} = [z_1, z_2, \ldots, z_C]^T$ into a vector of probabilities $\hat{\mathbf{y}}$:

$$\hat{y}_c = \text{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{2.10}$$

where each predicted probability satisfies $\hat{y}_c \in (0, 1)$ and the sum of all probabilities equals one: $\sum_{c=1}^{C} \hat{y}_c = 1$.
The loss function for multi-class classification is the **Cross-Entropy Loss**:

$$\ell = -\log \hat{y}_y \tag{2.11}$$

The softmax gives one probability per class $(\hat{y}_1, \ldots, \hat{y}_k)$ summing to 1, and only the probability of the true class y enters the loss.

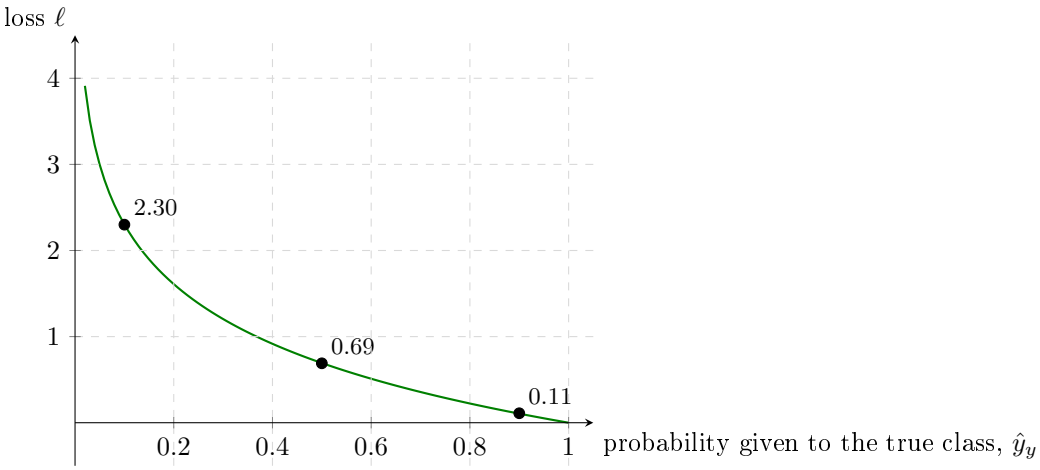


Figure 2.6: Multi-class cross-entropy loss $\ell = -\log \hat{y}_y$ with specific examples.

2.3 XOR Problem in Linear Model

Models that rely purely on a linear score followed by a fixed non-linear output function (like logistic regression) are known as *Generalized Linear Models*. Even though they apply a non-linear function (e.g., the S-shaped sigmoid curve), this function is strictly monotonic and only rescales the score. Consequently, the decision boundary,the region where the model decides to switch classes, remains a flat **hyperplane** ($z = 0$).
Consider the example in Fig 2.7, suppose a linear classifier $\hat{y} = 1 \iff \mathbf{w}^\top \mathbf{x} + b \geq 0$. Now considering the four point of the XOR:

- (0,0): $b < 0$
- (1,0): $w_1 + b \geq 0$
- (0,1): $w_2 + b \geq 0$
- (1,1): $w_1 + w_2 + b < 0$

we obtain the conditions for the parameters of the model, but if we combine the conditions for the intermediate points: $w_1 + w_2 + b + b \geq 0 \implies w_1 + w_2 + b \geq -b$ but from the first condition we know $b < 0$ so $w_1 + w_2 + b > 0$ but this is contradiction with the fourth condition. **No such w,b exists**.

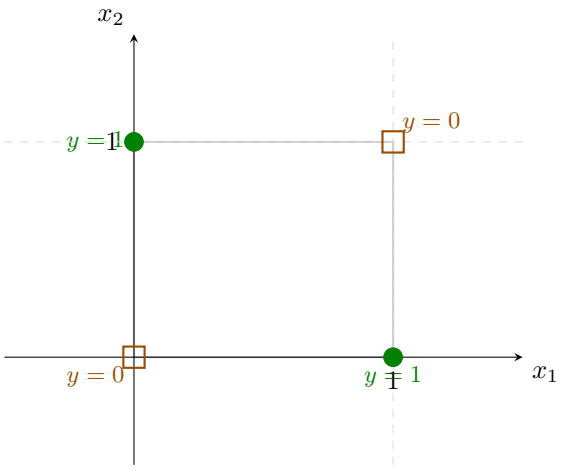


Figure 2.7: Geometric representation of the XOR problem showing linear non-separability.

The only fix to solve those type of problem it is to compute the score on new features $\phi(x)$. The **Representation** $\phi : \mathbb{R} \implies \mathbb{R}^{\mathbb{D}}$, turns the d raw inputs into D new features and then the model use those feature in the logit:

$$f(\mathbf{x}; \theta, \mathbf{w}) = \phi(\mathbf{x}; \theta)^{\top} \mathbf{w} + b$$

$\phi(x)$ can be obtained in three ways:

- **Generic** ϕ: a fixed rich map that poorly generalizes on hard problems
- **Hand designed** ϕ: written by experts, expensive and demanding
- **Learned** $\phi(\mathbf{x}; \theta)$: learned by gradient descent

Chapter 3

Deep Feedforward Networks

Also called multilayer perceptrons, a feedforward network is a composition of functions with a certain **Depth**(number of layers with parameters), **Width**(number of units dI in layer I). The goal of training is to choose θ so that

$$f(x;\theta) \approx f^*(x)$$

on new inputs, not only on training set.

3.1 Anatomy of a Unit and of a Layer

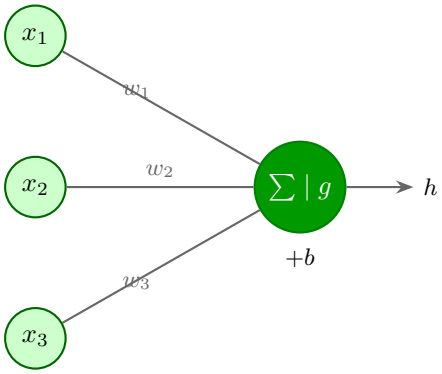
For one layer of m units (all read the same $\mathbf{x}$):

$$\mathbf{h} = g(\mathbf{W}^\top \mathbf{x} + \mathbf{b}), \quad \mathbf{x} \in \mathbb{R}^d,\ \mathbf{W} \in \mathbb{R}^{d\times m},\ \mathbf{b} \in \mathbb{R}^m \tag{3.1}$$

Column j of $\mathbf{W}$ holds the weights of unit j; g is applied to each entry separately.
$z = \mathbf{w}^\top \mathbf{x} + b$ is the **pre-activation**: a weighted sum plus bias. $h = g(z)$ is the **activation**; g is a fixed non-linear function, e.g. the ReLU $g(z) = \max\{0, z\}$.
As an example with $d = 3$ inputs and $m = 2$ ReLU units:

$$\mathbf{x} = (1, 0, 2), \quad \mathbf{W} = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b} = (0, -1)$$
$$\mathbf{z} = \mathbf{W}^\top \mathbf{x} + \mathbf{b} = (1 + 0 + 2, -1 + 0 + 2) + (0, -1) = (3, 0)$$
$$\mathbf{h} = \max\{0, \mathbf{z}\} = (3, 0)$$

For n examples at once: $\mathbf{H} = g(\mathbf{X}\mathbf{W} + \mathbf{1}\mathbf{b}^\top)$; row i of $\mathbf{H}$ is the layer output for example i.



Definition 1 (Saturation) *A unit is saturated when $|z|$ is so large that its output hardly changes: the slope $g'(z)$ is almost 0*

So when the slope is almost 0 the derivative is 0, is impossible to know from data how much the model has to change his parameters.

3.2 Multilayer Network in Matrix Form

The forward pass through an L-layer network is defined as:

$$\begin{aligned}\mathbf{h}^{(0)} &= \mathbf{x} \\ \mathbf{h}^{(l)} &= g(\mathbf{W}^{(l)\top}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 1, \ldots, L-1 \\ \hat{\mathbf{y}} &= o(\mathbf{W}^{(L)\top}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)})\end{aligned}$$

The last pre-activation, $\mathbf{z}^{(L)} = \mathbf{W}^{(L)\top}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$, is called the vector of **logits**; the output function o (identity, sigmoid or softmax) turns it into the prediction.

Definition 2 (Number of parameters) *Layer l has $d_{l-1} \cdot d_l$ weights and d_l biases:*

$$|\theta| = \sum_{l=1}^{L}(d_{l-1} + 1)d_l \tag{3.2}$$

Most parameters sit in the first layer. In Fig 3.1 there is an example of the calculus of parameters. The MLP is : $784 \to 256 \to 128 \to 10$. Input: $28 \times 28 = 784$ pixels. Output: 10 classes.

Layer	$\mathbf{W}^{(l)}$	biases	parameters
1	784×256	256	200 960
2	256×128	128	32 896
3 (output)	128×10	10	1 290
total			235 146

Table 3.1: Parameter count breakdown for the MNIST network architecture.

3.3 Solving XOR with NN

In the end of the last chapter we saw the limitations of linear model in the case of some problems like XOR. With NN this problem can be solved with one hidden layer as shown in Fig 3.2.

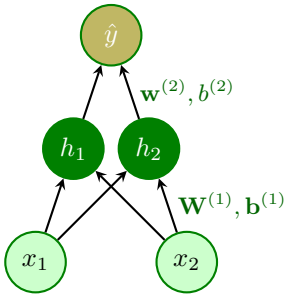


Figure 3.1: Neural network architecture for the XOR problem.

The forward pass is defined as:

$$\hat{y} = \mathbf{w}^{(2)\top} \max\left\{0, \mathbf{W}^{(1)\top}\mathbf{x} + \mathbf{b}^{(1)}\right\} + b^{(2)}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{w}^{(2)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b^{(2)} = 0$$

2 inputs, 2 hidden units, 1 output: $4 + 2 + 2 + 1 = 9$ parameters.

$$\underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}}_{\mathbf{X}} \longrightarrow \underbrace{\begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}}_{\mathbf{X}\mathbf{W}^{(1)}} \longrightarrow \text{add } \mathbf{b}^{(1)} \text{ to each row} \longrightarrow \underbrace{\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}}_{} \longrightarrow \underbrace{\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}}_{\max\{0,\cdot\}} \longrightarrow \underbrace{\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}}_{\times \mathbf{w}^{(2)} = \hat{\mathbf{y}}}$$

Figure 3.2: Step-by-step matrix representation of the XOR forward pass for an entire batch.

In $\mathbf{x}$-space no line separates the classes. In $\mathbf{h}$-space the two positive points coincide and a line does:

$$h_1 - 2h_2 = \frac{1}{2} \tag{3.3}$$

The output layer is still a **linear score**, computed on $\mathbf{h} = \phi(\mathbf{x})$ instead of $\mathbf{x}$.

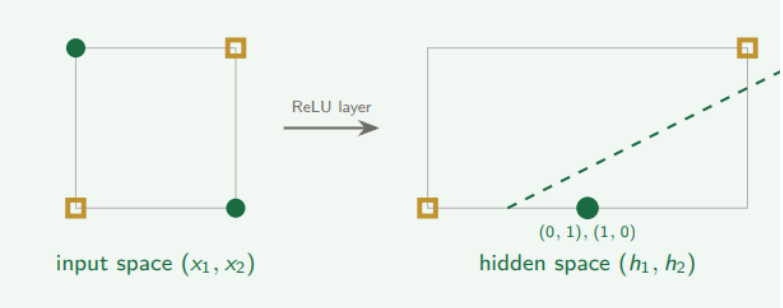


Figure 3.3: Difference between x-space and h-space

3.4 Hidden Unit as Feature Detectors

If we consider the *trigonometric* form of the pre-activation function $z = \mathbf{w}^{\mathbf{T}}\mathbf{x}+b = ||w||\,||x||cos\theta+b$ where θ is the angle between $\mathbf{w}$ and $\mathbf{x}$. For inputs of similar length, z is largest when $\mathbf{x}$ has the same direction of $\mathbf{w}$, so each hidden unit is a learned **template**. The bias estabilish how much strict must be the correspondence for the unit to be activated. In the example showed in Fig 3.4, the first layer units' become detectors of strokes (es. vertical lines)

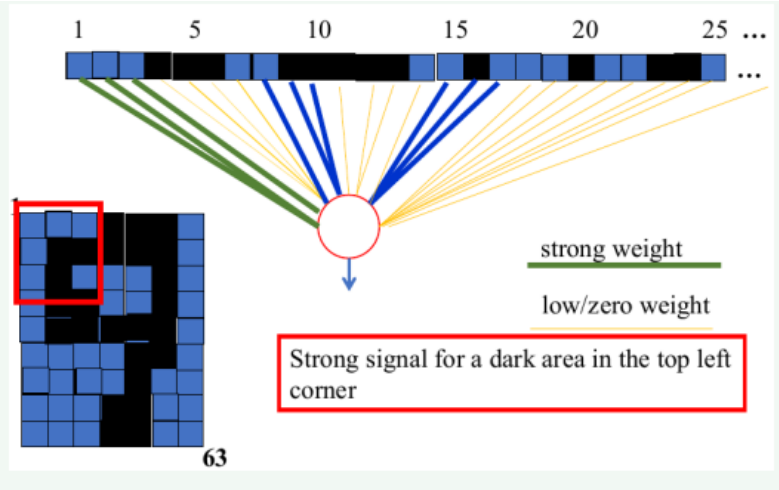


Figure 3.4: An 8x8 digit image flattened into 64 inputs

The limitation is that a unit is tied to fixed pixel position, so the same stroke elsewhere needs another unit.

3.5 Cost Functions and Output Units

3.5.1 Gradient-Based Learning

3.6 Notation

To maintain consistency throughout this document, we adopt the standard notation summarized below, largely following standard deep learning literature:

Symbol	Description
$x \in \mathbb{R}^d$	one input, with d features
y (or $\mathbf{y}$)	its target: a number, a class, or a vector
f^*	the unknown true function, $y = f^*(x)$
n	number of training examples
$(x^{(i)}, y^{(i)})$	the i-th training example
$X \in \mathbb{R}^{n \times d}$	all inputs, one example per row
m	minibatch size (examples per update)
k	number of classes

Table 3.2: Data notation summary.

Symbol	Description
L	number of layers (the depth)
d_l	number of units in layer l (its width); $d_0 = d$
$W^{(l)}, b^{(l)}$	weight matrix ($d_{l-1} \times d_l$) and bias vector of layer l
$z^{(l)}$	pre-activation: $W^{(l)\top} h^{(l-1)} + b^{(l)}$
$h^{(l)} = g(z^{(l)})$	activation of layer l; $h^{(0)} = x$
g	activation function of hidden units
o	output function (identity, sigmoid or softmax)
θ	all weights and biases together

Table 3.3: Network notation summary.

Note on indexing: Superscripts in parentheses, such as $^{(l)}$ or $^{(i)}$, always denote indices (referring to layers or training examples, respectively) and are never used as mathematical powers. Vectors are represented in bold lowercase letters, matrices in bold uppercase, and scalars in standard font.

Symbol	Description
$\hat{y} = f(x; \theta)$	prediction of the network
$\ell(\hat{y}, y)$	loss on one example
$J(\theta)$	cost: average loss on the training set
$\nabla_\theta J$	gradient: vector of all $\partial J / \partial \theta_j$
η	learning rate (step size)
logit z	the score before the sigmoid or softmax

Table 3.4: Learning and optimization notation summary.

Symbol	Description
$\sigma(z) = \frac{1}{1+e^{-z}}$	logistic sigmoid, values in $(0, 1)$
$\zeta(a) = \log(1 + e^a)$	softplus: a smooth version of $\max\{0, a\}$
$e_y \in \{0, 1\}^k$	one-hot vector: 1 in position y, 0 elsewhere
$\mathbb{I}[\text{cond}]$	indicator: 1 if cond is true, else 0
$\mathbf{1}$	vector of ones
$\odot$	element-wise product

Table 3.5: Functions and symbols notation summary.

Chapter 4

Exercises

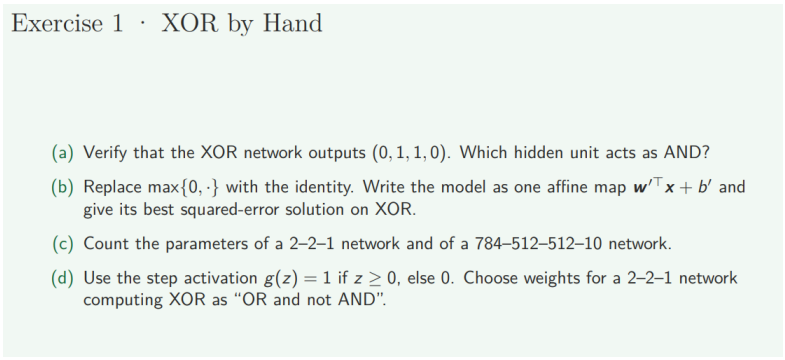
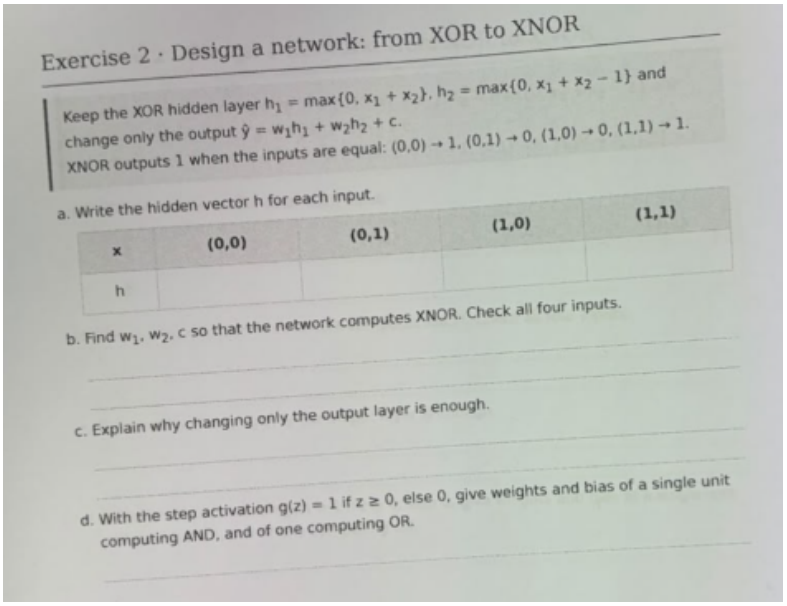


Figure 4.1: exe1



Exercise 2 · Design a network: from XOR to XNOR

Keep the XOR hidden layer $h_1 = \max\{0, x_1 + x_2\}$, $h_2 = \max\{0, x_1 + x_2 - 1\}$ and change only the output $\hat{y} = w_1 h_1 + w_2 h_2 + c$.
XNOR outputs 1 when the inputs are equal: $(0,0) \to 1$, $(0,1) \to 0$, $(1,0) \to 0$, $(1,1) \to 1$.

a. Write the hidden vector h for each input.

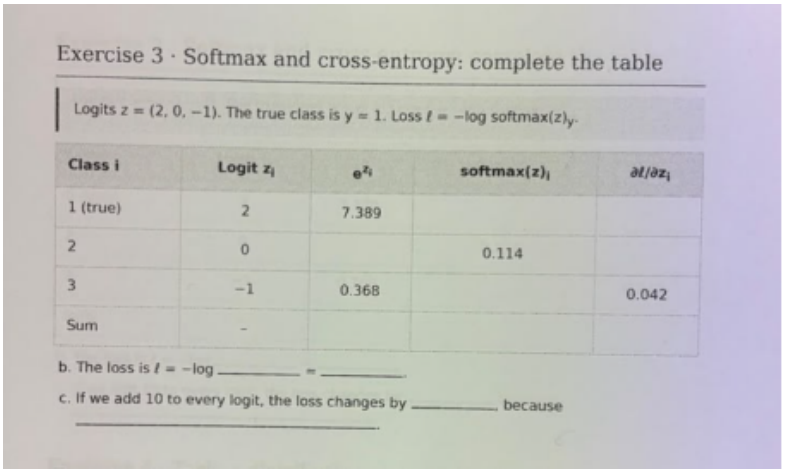
x	(0,0)	(0,1)	(1,0)	(1,1)
h				

b. Find w_1, w_2, c so that the network computes XNOR. Check all four inputs.

c. Explain why changing only the output layer is enough.

d. With the step activation $g(z) = 1$ if $z \geq 0$, else 0, give weights and bias of a single unit computing AND, and of one computing OR.

Figure 4.2: exe2



Exercise 3 · Softmax and cross-entropy: complete the table

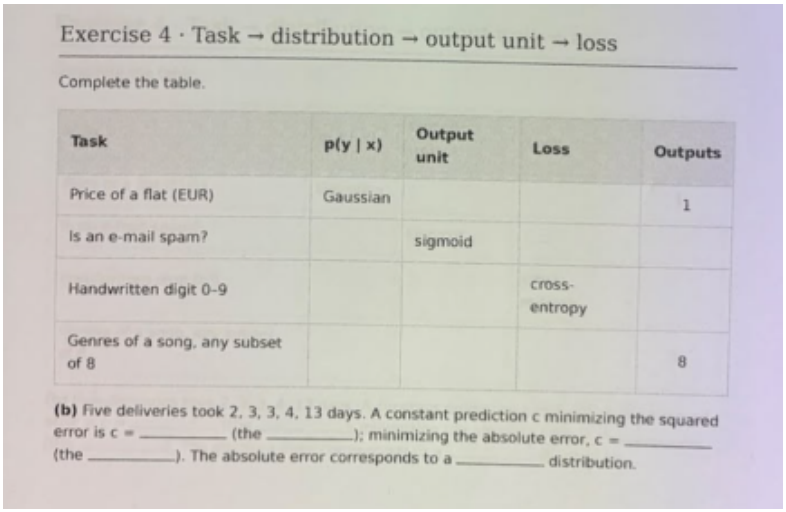
Logits $z = (2, 0, -1)$. The true class is $y = 1$. Loss $\ell = -\log \text{softmax}(z)_y$.

Class i	Logit z_i	e^{z_i}	$\text{softmax}(z)_i$	$\partial \ell / \partial z_i$
1 (true)	2	7.389		
2	0		0.114	
3	−1	0.368		0.042
Sum	–			

b. The loss is $\ell = -\log$ _____ = _____.

c. If we add 10 to every logit, the loss changes by _____ because _____.

Figure 4.3: exe3



Exercise 4 · Task → distribution → output unit → loss

Complete the table.

Task	p(y | x)	Output unit	Loss	Outputs
Price of a flat (EUR)	Gaussian			1
Is an e-mail spam?		sigmoid		
Handwritten digit 0–9			cross-entropy	
Genres of a song, any subset of 8				8

(b) Five deliveries took 2, 3, 3, 4, 13 days. A constant prediction c minimizing the squared error is $c =$ _____ (the _____); minimizing the absolute error, $c =$ _____ (the _____). The absolute error corresponds to a _____ distribution.

Figure 4.4: exe4